

Data mining & machine learning

3. Time series

Francesco Pistolessi

University of Pisa

`francesco.pistolessi@unipi.it`

M.Sc. in Artificial Intelligence and Data Engineering



UNIVERSITÀ DI PISA

By the end of this lecture, you will be able to:

- explain what a **time series** is and why it differs from tabular data;
- represent and index time series in **pandas**;
- **slice and subset** a time series by date;
- generate **date ranges** and work with frequencies;
- **resample**: downsample and upsample a series;
- handle **missing values** in time series (ffill, bfill, interpolation);
- apply **moving window** functions for smoothing.

Main topics

what is a time series?

datetime & pandas

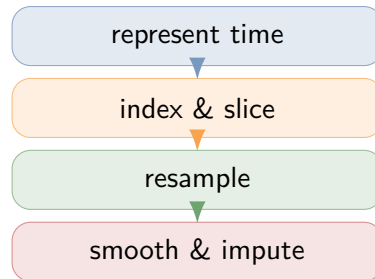
resampling

missing values

moving windows

Notebook structure

- 1 Date and time data types
- 2 Time series basics in pandas
- 3 Date ranges, frequencies, shifting
- 4 Time zone handling
- 5 Resampling and frequency conversion
- 6 Moving window functions



What is a time series?

In real world we have continuous values that we have to sample to obtain discrete values

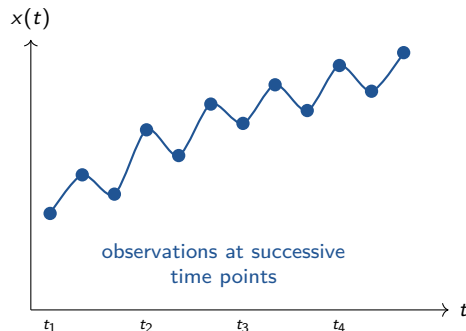
A **time series** is a sequence of observations collected **over time**: the *order* carries information.

Time series are everywhere:

- **Medicine**: ECG, EEG, wearable sensors
- **Finance**: stock prices, exchange rates
- **Meteorology**: temperature, rainfall, wind speed
- **Industry**: machine sensor readings

Key difference from tabular data

In a standard table, rows are independent observations. In a time series, each value relates to what came before and after it. Time is not a label.



Three ways to associate data with time

Timestamp

A specific **instant** in time.

2024-03-15 14:30:00

Used when you record an event at a precise moment (sensor reading, transaction).

Period

An entire **interval** treated as one unit.

Q1-2024, 2024-03

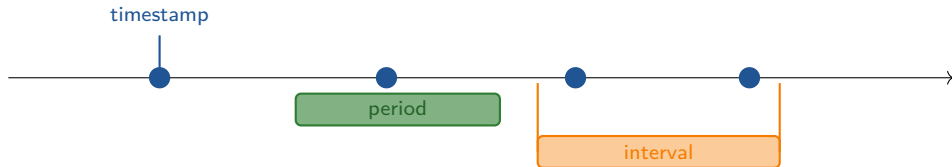
Used for aggregated data: monthly sales, annual revenue, daily totals.

Interval

A span defined by a **start** and **end** timestamp.

[2024-03-01, 2024-03-31]

Used when duration matters (hospital stay, project phase).



Fixed vs. irregular frequency

Fixed frequency

Observations arrive at **regular intervals**: every second, every hour, every business day.

- Easy to model and forecast
- Supports efficient indexing
- Generated with `pd.date_range`

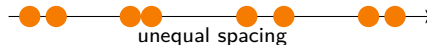


pandas supports both

Irregular frequency

Observations arrive at **uneven intervals**: user clicks, sensor events, medical records.

- More realistic in practice
- Harder to model directly
- Often converted to fixed frequency via **resampling**



resample converts between them

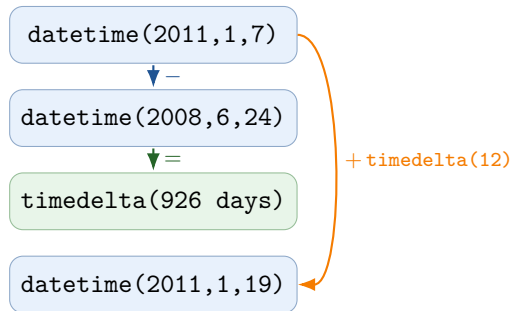
Python's datetime object

Python's standard library provides `datetime`, which stores a complete date and time down to microsecond precision. The most important types:

- `datetime` — date *and* time
- `date` — date only
- `time` — time only
- `timedelta` — a **duration** (difference between two datetimes)

Why `timedelta`?

To do arithmetic: add 12 days, subtract 3 months, compute hours between events. `timedelta` can also be added to shift a date.



Working with datetime and timedelta

```
from datetime import datetime, timedelta

# Current date and time
now = datetime.now()
now.year, now.month, now.day, now.microsecond

# Difference between two datetimes -> timedelta
delta = datetime(2011, 1, 7) - datetime(2008, 6,
    24, 8, 15)
delta          # timedelta(926 days, 56700 seconds)
delta.days     # 926
delta.seconds  # 56700

# Shift a date forward or backward
start = datetime(2011, 1, 7)
start + timedelta(12)      # 2011-01-19
start - 2 * timedelta(12)  # 2010-12-14
```

Output

- `delta.days` → 926
- `delta.seconds` → 56700
- `start + timedelta(12)`
→ 2011-01-19

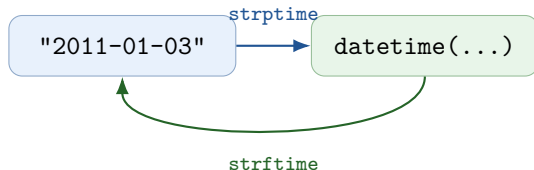
`timedelta` accepts:
days, seconds, microseconds,
milliseconds, minutes,
hours, weeks.

Converting between strings and datetime

Real-world data often stores dates as **strings**.

Python offers two complementary functions:

- `strptime`: **string parse**: `string` $\rightarrow$ `datetime`
- `strftime`: **string format**: `datetime` $\rightarrow$ `string`



```
from datetime import datetime
# String -> datetime
datetime.strptime(
    "2011-01-03", "%Y-%m-%d")
# datetime(2011, 1, 3, 0, 0)
# datetime -> string
stamp = datetime(2011, 1, 3)
stamp.strftime("%Y-%m-%d")
# '2011-01-03'
```

Common codes:

%Y year	%m month	%d day
%H hour	%M minute	%S second

Parsing datetimes with pandas: `pd.to_datetime`

```
import pandas as pd

datestrs = ["2011-07-06 12:00:00",
            "2011-08-06 00:00:00"]

# Parse a list of strings -> DatetimeIndex
pd.to_datetime(datestrs)
# DatetimeIndex(['2011-07-06 12:00:00',
#                '2011-08-06 00:00:00'],
#               dtype='datetime64[ns]')

# None becomes NaT (Not a Time)
idx = pd.to_datetime(datestrs + [None])
idx[2]      # NaT
pd.isna(idx) # [False, False, True]
```

`pd.to_datetime`

- Accepts lists, Series, strings
- Handles many formats automatically
- Returns a `DatetimeIndex`
- None or NaN → **NaT**

NaT

NaT (Not a Time) is the datetime equivalent of NaN. Use `pd.isna()` to detect it.

A time series in pandas

In pandas, a time series is a Series (or DataFrame) whose **index is a DatetimeIndex**.

The index is stored as `datetime64[ns]`: a 64-bit integer counting nanoseconds since 1970-01-01.

DatetimeIndex properties

- `ts.index` — the index itself
- `ts.index.dtype` — `datetime64[ns]`
- `ts.index[0]` — a Timestamp object

```
from datetime import datetime
import pandas as pd, numpy as np

dates = [datetime(2011,1,2),
          datetime(2011,1,5),
          datetime(2011,1,7),
          datetime(2011,1,8),
          datetime(2011,1,10),
          datetime(2011,1,12)]

ts = pd.Series(
    np.random.standard_normal(6),
    index=dates)
```

```
2011-01-02    -0.204
2011-01-05     0.478
2011-01-07    -0.519
2011-01-08    -0.555
2011-01-10     1.966
2011-01-12     1.394
dtype: float64
```

Indexing and slicing a time series

pandas time series support very **flexible indexing**:

- A Timestamp or datetime object
- A **string** (pandas parses it automatically)
- A **partial string** for year or year-month selection
- Standard Python **slice** syntax with dates

Slices are views, not copies

Slicing a time series returns a **view** of the original data. Modifying the slice also modifies the original. Use `.copy()` if you need an independent copy.

```
# Single value by string
ts["2011-01-10"] # 1.966

# All of year 2001
longer_ts["2001"]

# One month
longer_ts["2001-05"]

# Slice between two dates
ts[datetime(2011,1,7):]
ts["2011-01-06":"2011-01-11"]
# Endpoint is INCLUSIVE
# (unlike Python list slicing)
```

string indexing

partial date works

endpoint inclusive

Duplicate timestamps

Sometimes multiple observations share the same timestamp (e.g., multiple trades in the same second).

pandas allows this: the `DatetimeIndex` does not need to be unique. Check with `ts.index.is_unique`.

When you index a duplicated timestamp, pandas returns a **Series** of all matching rows (not a scalar).

Aggregating duplicates

Use `groupby(level=0)` to collapse duplicate timestamps. Common operations: `mean()`, `sum()`, `count()`.

```
dates = pd.DatetimeIndex([
    "2000-01-01", "2000-01-02",
    "2000-01-02", "2000-01-02",
    "2000-01-03"])
dup_ts = pd.Series([0,1,2,3,4],
                    index=dates)

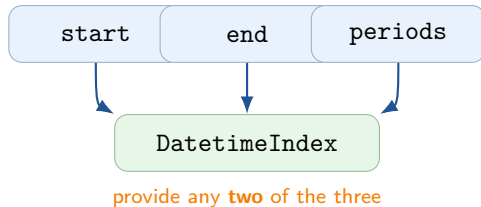
dup_ts.index.is_unique    # False

dup_ts["2000-01-03"]      # scalar: 4
dup_ts["2000-01-02"]      # Series: 1,2,3

grouped = dup_ts.groupby(level=0)
grouped.mean()
# 2000-01-01    0.0
# 2000-01-02    2.0
# 2000-01-03    4.0
```

Generating date ranges: `pd.date_range`

`pd.date_range` generates a `DatetimeIndex` with a **fixed frequency**. Provide any two of: start, end, periods (i.e., how many dates to generate).



```
# Start + end (daily by default)
pd.date_range("2012-04-01",
              "2012-06-01")
# Start + number of periods
pd.date_range(start="2012-04-01",
              periods=20)
# End + number of periods
pd.date_range(end="2012-06-01",
              periods=20)
# Business month end frequency
pd.date_range("2000-01-01",
              "2000-12-01", freq="
              BM")
# Normalize time to midnight
pd.date_range("2012-05-02 12:56:31"
              ,
              periods=5, normalize=
              True)
```

Frequency aliases and composite offsets

pandas uses **string aliases** for frequencies.
Prepend a multiplier: "4H" = every 4 hours.
Offsets can be **combined** with addition.

Alias	Meaning
D	Calendar day
B	Business day
H	Hour
T / min	Minute
S	Second
M	Month end
BM	Business month end
W-MON	Weekly, anchored Monday
WOM-3FRI	3rd Friday of each month

```
from pandas.tseries.offsets import Hour,
    Minute

Hour(4)                                # <4 * Hours>
Hour(2) + Minute(30)                   # <150 * Minutes>

# Every 4 hours
pd.date_range("2000-01-01",
              "2000-01-03 23:59", freq="4H")

# Every 1h 30min
pd.date_range("2000-01-01",
              periods=10, freq="1h30min")

# 3rd Friday of each month
pd.date_range("2022-01-01",
              "2022-09-01", freq="WOM-3FRI")
# [2022-01-21, 2022-02-18, ...]
```

Shifting a time series

Shifting moves the *values* of a series forward or backward in time while **keeping the index fixed**. This introduces NaN at one end.

A key application: percent change

$$\text{pct_change}(t) = \frac{x(t)}{x(t-1)} - 1$$

`ts / ts.shift(1) - 1` computes this for the whole series.

Passing `freq` to `shift` moves the **index** instead of the values (useful to advance timestamps).

Original

Apr	val4
Mar	val3
Feb	val2
Jan	val1

shift(2)

Apr	val2
Mar	val1
Feb	NaN
Jan	NaN

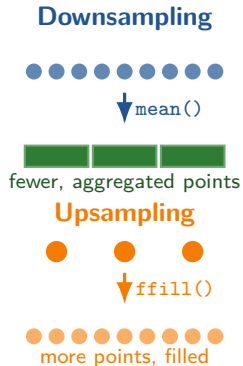
```
ts.shift(2)           # forward 2 steps
ts.shift(-2)          # backward 2 steps
ts / ts.shift(1) - 1  # pct change
ts.shift(2, freq="M") # shift the INDEX
```


Resampling: the big picture

Resampling converts a time series from one frequency to another using `ts.resample(freq)`.

Two directions:

- **Downsampling** — reducing frequency (e.g., minutes → hours). You *aggregate* many observations into one: `sum()`, `mean()`, `count()`, `ohlc()`.
- **Upsampling** — increasing frequency (e.g., weekly → daily). You *fill in* new timestamps that had no observation.



```
# 1-minute series (12 points: 0..11)
dates = pd.date_range("2000-01-01",
                      periods=12, freq="T")
ts = pd.Series(np.arange(12), index=dates)

# Sum within each 5-minute bin
ts.resample("5min").sum()
# 2000-01-01 00:00    10    (0+1+2+3+4)
# 2000-01-01 00:05    35    (5+6+7+8+9)
# 2000-01-01 00:10    21    (10+11)

# Control which side of each bin is closed
ts.resample("5min", closed="right").sum()

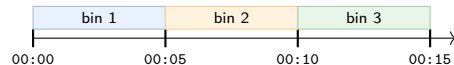
# Control which endpoint labels the bin
ts.resample("5min", closed="right",
            label="right").sum()
```

closed and label

Each bin $[a, b)$ raises an ambiguity:

- `closed="left"` (default): $[a, b)$
- `closed="right"`: $(a, b]$

`label` controls which endpoint appears in the output index.



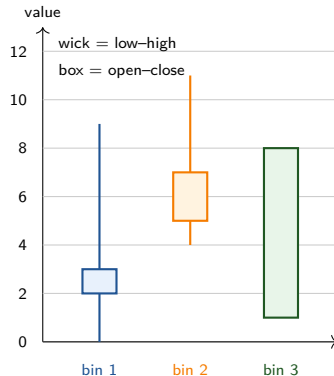
OHLC aggregation

```
# Shuffle the values for a realistic example
ts = pd.Series(
    np.random.permutation(np.arange(12)),
    index=dates)
ts.resample("5min").ohlc()
#           open  high  low  close
# 00:00         3     9    0      2
# 00:05         5    11    4      7
# 00:10         1     8    1      8
```

OHLC — finance standard

- **Open** — first value in bin
- **High** — maximum
- **Low** — minimum
- **Close** — last value

Standard for financial candlestick charts.



Upsampling

```
# Weekly DataFrame (2 rows)
frame = pd.DataFrame(
    np.random.standard_normal((2, 4)),
    index=pd.date_range("2000-01-01",
                        periods=2,
                        freq="W-WED"),
    columns=["Colorado", "Texas",
            "New York", "Ohio"])

# Upsample to daily frequency
# New dates have NaN values
df_daily = frame.resample("D").asfreq()
# 2000-01-05    0.47 -0.13 -1.02  0.31
# 2000-01-06     NaN     NaN     NaN     NaN
# 2000-01-07     NaN     NaN     NaN     NaN
# ...
```

Upsampling is not free

Downsampling summarises real measurements.

Upsampling creates values that were never measured. They must be **filled in** (ffill, interpolation...) as an approximation.



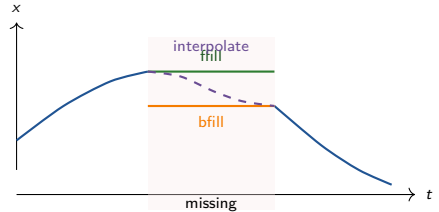
Missing values in time series

Missing values are common in time series: sensors fail, measurements skipped, data arrive late.

Standard strategies like *mean imputation* are often **inappropriate**: they use future information or destroy temporal structure.

The lookahead problem

A **lookahead** occurs when future data is used to fill a past gap. This is fine for retrospective analysis, but it *leaks information* if you are building a forecasting model — the model will appear better than it really is at prediction time.



Forward fill, backward fill, interpolation

Forward fill (ffill)

LOCF: Last Observation Carried Forward.

The last known value is repeated until the next observation.

```
ts.ffill()
```

```
ts.ffill(limit=10)
```

Safe for forecasting: only uses past values.

Backward fill (bfill)

NOCB: Next Observation Carried Backward.

The next known value is propagated into the past.

```
ts.bfill()
```

Lookahead: uses future data. Only appropriate when forecasting is not the goal.

Interpolation

Estimates missing values assuming a geometric relationship.

```
ts.interpolate('linear')
```

```
ts.interpolate('quadratic')
```

```
ts.interpolate('polynomial',  
order=7)
```

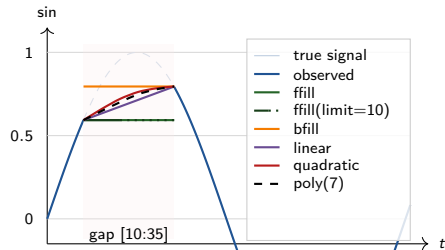
Uses past *and* future $\Rightarrow$ **lookahead**.

ffill: safe for forecasting

bfill / interpolate: lookahead

Imputation in practice: a sin-wave example

```
import math
# Clean sin wave (100 samples, 1 per second)
sin_fun = [np.sin(x/math.pi/5)
            for x in range(100)]
drange = pd.date_range("2000-01-01",
                        freq='S', periods=100)
sin_frame = pd.Series(sin_fun, drange)
# Save original, then introduce a gap
original = sin_frame.copy(deep=True)
sin_frame[10:35] = np.nan
# Apply the three strategies
sin_frame.ffill()
sin_frame.ffill(limit=10)
sin_frame.bfill()
sin_frame.interpolate('linear')
sin_frame.interpolate('quadratic')
sin_frame.interpolate('polynomial',
                        order=7)
```



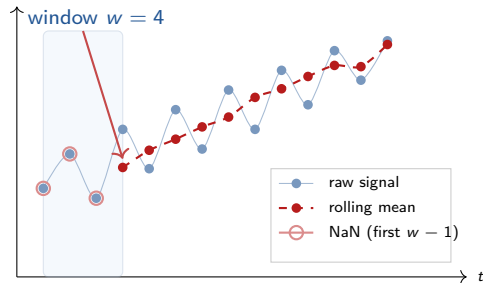
Moving window functions: the idea

A **moving window** (rolling window) function computes a statistic over a sliding sub-sequence of fixed length w .

At each t , the window contains the w most recent values. Result: new series of the same length, with NaN for the first $w - 1$ positions.

Why use moving windows?

- **Smoothing:** remove high-frequency noise while preserving the underlying trend.
- **Imputation:** fill gaps with local averages (better than ffill when the series is noisy).
- **Feature engineering:** rolling mean, min, max are strong features for models on time series.



Rolling windows with pandas

```
# Load stock data (AAPL, MSFT, XOM)
close_px = pd.read_csv("dataset/stock_px.csv",
    parse_dates=True, index_col=0)
close_px = close_px[["AAPL", "MSFT", "XOM"]]

# Resample to business day, forward-fill gaps
close_px = close_px.resample("B").asfreq().ffill()

# 250-day rolling mean (long-term trend)
close_px["AAPL"].rolling(250).mean()

# 30-day rolling mean (short-term trend)
close_px["AAPL"].rolling('30D').mean()

# First (window-1) values are NaN:
# not enough data to fill the first window
```

rolling() options

- window — integer or offset string
- center=True — label at centre of window (introduces lookahead)
- min_periods — minimum non-NaN values required

By default, labels are placed at the **right** edge of the window (past values only). center=True uses both past and future — a lookahead.

Effect of window size on smoothing

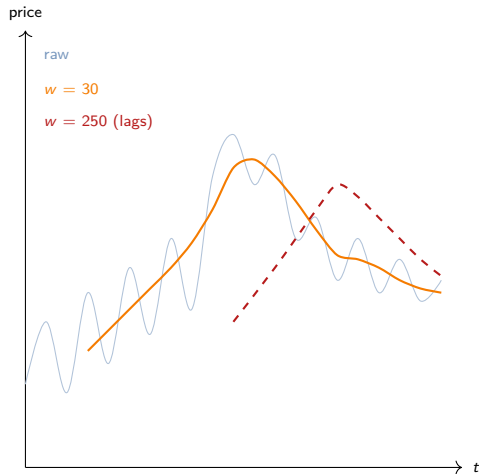
The window size w is the key trade-off:

Small w (e.g., 30 days): the moving average stays close to the original signal. Noise is reduced but not eliminated. Local peaks and valleys are still visible.

Large w (e.g., 250 days): the curve becomes very smooth but **lags** behind the original signal and loses local detail. Sudden changes are hidden.

Remember

A wider window reduces noise but also reduces information. Choose w based on the time scale of the features you care about.



The Savitzky–Golay filter

```
from scipy.signal import savgol_filter

# Apply SG filter: window=100, degree=4
close_px["AAPL_sg"] = savgol_filter(
    close_px["AAPL"],
    window_length=100,
    polyorder=4)

# Compare the three signals
close_px["AAPL"].plot(label="original")
close_px["AAPL"]\
    .rolling(100, center=True)\
    .mean().plot(label="simple MA (w=100)")
close_px["AAPL_sg"].plot(
    label="Savitzky-Golay")
plt.legend()
plt.title("Apple: MA vs Savitzky-Golay")
plt.show()
```

What is Savitzky–Golay?

Instead of averaging within the window, SG fits a **polynomial** of degree `polyorder` and evaluates it at the centre point.

This **preserves peaks and edges** better than simple averaging, because the polynomial follows local curvature.

Requires two parameters: `window_length` (odd integer) and `polyorder` (degree of the fitting polynomial).

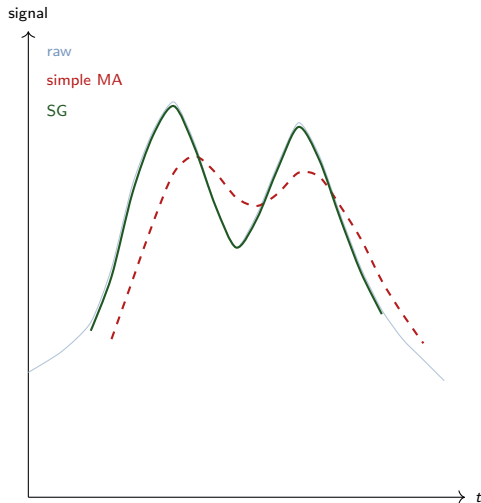
Simple MA vs. Savitzky–Golay: key difference

With the **same window length** (100 periods):

- The simple MA **flattens peaks**: it averages away local extremes.
- SG fits a local polynomial that tracks the signal shape more faithfully, so peaks are better preserved.

When to use each

- **Simple MA**: general trend, financial indicators, easy to interpret.
- **Savitzky–Golay**: scientific or sensor data where peak location matters (spectroscopy, ECG, accelerometers).



- McKinney, W. (2017). *Python for Data Analysis*, Chapter 11. O'Reilly Media. (*primary reference for this notebook*)
- Nielsen, A. (2019). *Practical Time Series Analysis: Prediction with Statistics & Machine Learning*. O'Reilly Media.
- pandas documentation — Time series and date functionality:
https://pandas.pydata.org/docs/user_guide/timeseries.html
- Savitzky, A. and Golay, M.J.E. (1964). *Smoothing and Differentiation of Data by Simplified Least Squares Procedures*. Analytical Chemistry.

Conclusions

Core ideas

- A time series is a sequence of observations where **order matters**.
- pandas represents time series as Series/DataFrame with a DatetimeIndex.
- **resample** converts between frequencies; **shift** moves values in time.

represent time correctly

resample with care

never leak the future

Methodological message

- Imputation strategy matters: **ffill is safe**; bfill and interpolation introduce lookahead.
- Rolling windows smooth noise but trade resolution for smoothness.
- Savitzky–Golay preserves peaks better than simple averaging.

Questions?